- Add provider specific Spring Boot starter module to the project dependencies.
  - Example: Maven dependency for Vertex AI Gemini

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-vertex-ai-gemini-spring-boot-starter</
artifactId>
</dependency>
```

- Define configuration entries to specify model end point details.
  - Sample configuration (in *application.properties*) for Google Gemini model.

```
# Gemini Vertex AI configuration
spring.ai.vertex.ai.gemini.project-id=${DEFAULT_PRJ}
spring.ai.vertex.ai.gemini.location=us-central1
spring.ai.vertex.ai.gemini.chat.options.model=gemini-1.5-flash
```

- Use various abstractions provided by Spring AI to interact with the AI model.
  - *ChatModel*: For chatting and text generation along with multi-modality [Text. Media to text].
  - *ImageModel:* For generating images [Text to image].
  - *AudioModel:* For transcription and text-to-speech use cases.
  - *EmbeddingModel:* For generating embeddings from text, images and videos.
- Based on the dependencies available in the class-path, Spring AI performs the necessary plumbing to instantiate model provider-specific classes and make them available to code via generic interfaces.

```
private final ChatClient chatClient;
private final ChatModel chatModel;

@Autowired
public SpringAIController(ChatModel chatModel) {
    chatClient = ChatClient.builder(chatModel).build();
    this.chatModel = chatModel;
}
```

Since Gemini-spring-boot-starter is available in the class-path, Spring AI automatically creates an instance of VertexAiGeminiChat and makes it available via the ChatModel interface in the above code.

Only organisations that can pivot quickly, with minimal effort, will be able to survive in this dynamic and competitive environment. Spring AI provides the flexibility to teams that want to switch from one model provider to another. With all the abstraction layers provided by the Spring AI framework, changing a model provider is possible with just configuration changes — no code changes are needed.

The code examples given above use Google Gemini as the backend model. Switching to another model provider like Ollama is as simple as changing the configuration. A sample configuration for using Ollama with Spring AI is given below.

1. Replace Google Gemini Maven dependency with Ollama starter module, as shown below.

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-ollama-spring-boot-starter</
artifactId>
</dependency>
```

2. In the *application.properties* configuration file, specify the Ollama end point and model to be used.

```
#Ollama Configuration
spring.ai.ollama.base-url=http://localhost:11434
spring.ai.ollama.chat.options.model=tinyllama
```

With this configuration, Spring AI will automatically initialise Ollama-specific classes for ChatModel, etc, and make them available.

Implementing GenAI use cases using Spring AI

Let's look at how to use Spring AI code and classes to implement and integrate various GenAI use cases into applications. The sample code and the responses given below are based on using Google as the model provider and Gemini-flash model as shown in the configuration section above.

Text and language handling: Implementing a text or language based GenAI use case usually implies either processing text or a QA chat scenario. Both can be done using a Spring AI ChatClient interface as depicted in the code below. Consider the following example use case where we ask the model to explain the binary search algorithm in five points. The Java code using Spring AI is:

```
String userPrompt = "Explain binary search algorithm in 5
bullet points";
Prompt prompt = new Prompt(userPrompt);
String response = chatClient.prompt(prompt).call().content();
```

Sample output from the above code is:

```
•  Here's a breakdown of the binary search in five bullet
   points.
•  Sorted data: Binary search works only on sorted data
   (ascending or descending).
```